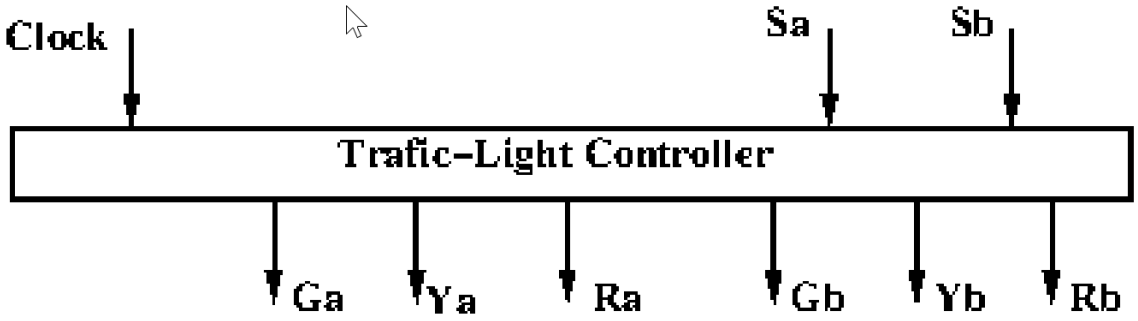
I utilized Yosys 0.33 and its built-in ABC mapper onto the OSU 0.5 µm standard-cell library.

I ran two models, both utilizing the Kogge-Stone adder, which is widely regarded as one of the fastest adder designs. The first approach, in ALU_16Bit_TopLevel, simply used a shared Kogge-Stone adder, where a mux picks the adder's second input (B, ~B, 0, or FFFF) based on alu_code. The second approach, in Parallel_ALU, used a parallel Kogge-Stone adder for each arithmetic operation (A + B, A - B, A + 1, and A - 1), all computing at once, with alu_code only selecting which result to output at the end.

Based on the synthesis, I found that the worst case path of the shared model runs from alu_code[3] to overflow, with a delay of 4.17 ns through 16 gates. The path first decodes alu_code into the select signal for the y mux and buffers it out to all 16 bits. It then passes through the y mux and the propagate XOR (A ^ y), through the Kogge-Stone carry tree up to bit 15, and ends at the signed overflow logic. About 1.7 ns of the delay is spent decoding and buffering alu_code before the addition even starts, while the adder itself takes about 1.8 ns. In the parallel model, the worst case path instead runs from A[1] to overflow, with a delay of 2.37 ns through 9 gates, since alu_code is no longer in front of any adder. This cuts the worst case delay by about 43%, at the cost of about 29% more area (284,850 vs 221,391) from the three extra adders.

Problem 3



a)Design the traffic controller and show that the light controller is safe. Describe in detail why the design is safe.

I designed the controller with safety in mind in a few key ways. The first and most important design decision was to limit the states to 4 such that there only exists safe states, and all outputs are stored in a register. If by any series of events, the outputs become harmful, the traffic light enters the flashing red error mode and does not leave unless reset. Under no normal circumstances can the circuit enter an unsafe state, and if it does, it moves to the error state. The only issue with this design is that a transition between two safe states can be unsafe. If for example the yellow light stage is skipped, an accident could occur. While this is theoretically possible with my design, it would require a bit to randomly flip from state 1 to state 3 or vise versa.

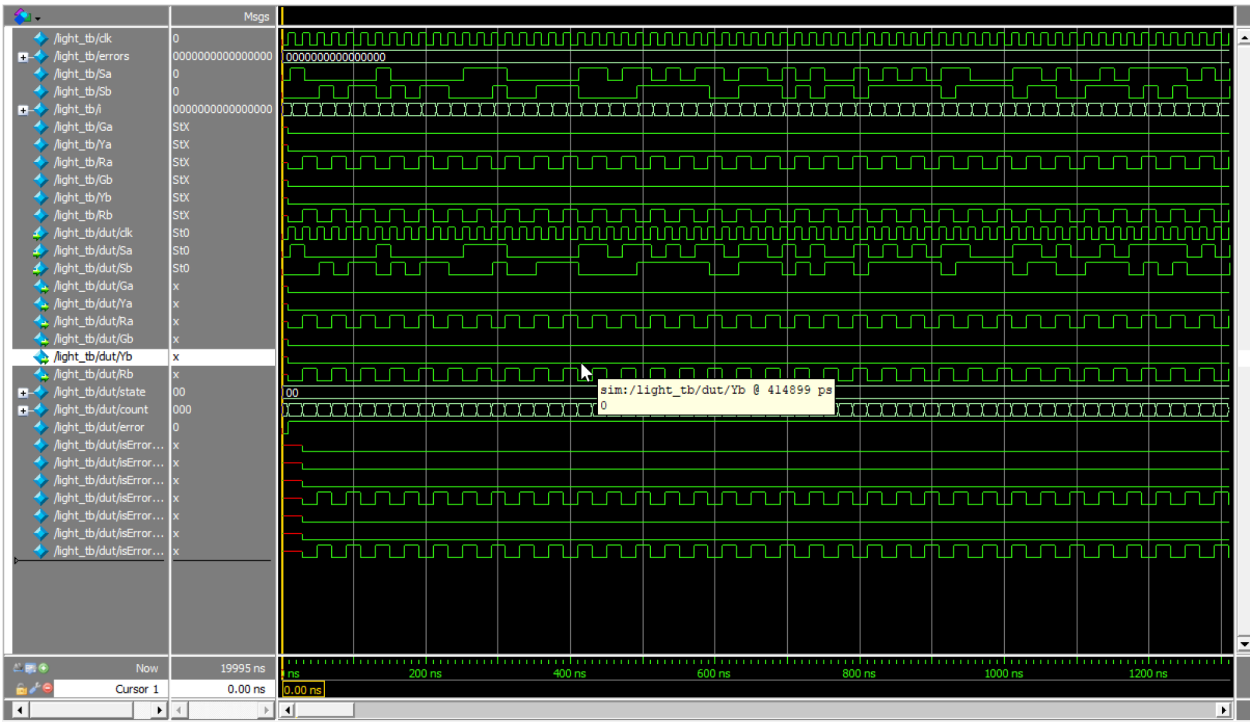
This could be fixed by tracking previous states. If the states do not follow a counting pattern (1,2,3,4), then the circuit could be put into error mode.

b) Write a Verilog model for the traffic light controller.

The model is written and called TrafficLightController.v

c) Simulate your model using ModelSim to verify correctness and safety of your design.

The model's waveform was viewed to confirm it conformed to all logic requirements. The model was tested against a testbench that looked for any unsafe outputs and it received zero errors. The model was also synthesized. All of this information is below:




```
=== Resource usage ===
Info (21058): Implemented 3 input pins
Info (21059): Implemented 6 output pins
Info (21061): Implemented 35 logic cells

Full report: C:\Users\lukes\DocumentsReal\ECSE318\Github\ecse318\Labs\Lab2\P2\synth_TrafficLightController\TrafficLightController.map.rpt
PS C:\Users\lukes\DocumentsReal\ECSE318\Github\ecse318\Labs\Lab2\P2>
```

Problems 5 Output Debug Console Terminal Ports

```
Reading pref.tcl

# 2020.1

# vsim -c -sv_seed 42 -do "run -all; quit -f" light_tb
# Start time: 12:25:55 on Oct 04,2026
# Loading work.light_tb
# Loading work.TrafficLightController
# run -all
# Errors: 0
# ** Note: $finish    : light_tb.v(31)
#    Time: 19995 ns  Iteration: 0  Instance: /light_tb
# End time: 12:25:55 on Oct 04,2026, Elapsed time: 0:00:00
# Errors: 0, Warnings: 0
PS C:\Users\lukes\DocumentsReal\ECSE318\Github\ecse318\Labs\Lab2\P2>
```